

Ring signatures for anonymous self-credentials

Sophia Zhou
sophia.zhou@yale.edu

Advisor: Fan Zhang
f.zhang@yale.edu

*A Senior Thesis as a partial fulfillment of requirements
for the Bachelor of Science in Computer Science*

Department of Computer Science
Yale University
Dec 11, 2025

Acknowledgements

I want to express my gratitude to my mentors and professors, who have supported and nurtured my studies in computer science. Thank you to my advisors Prof. Fan Zhang and Giannis Kaklamanis, Prof. Sohee Park for organizing the CPSC 4900 class, Prof. Katerina Sotiraki for a great first class in cryptography, and everyone else along the way.

Contents

1	Introduction	5
2	Background	6
3	Methodology	9
4	Results	10
4.1	LRS-based ASC construction	10
4.2	Benchmarking results	11
5	Related Work	15
6	Conclusions	16
7	Future Work	17
A	Appendix	20
A.1	Codebase	20
A.2	Additional plots	20

Ring signatures for anonymous self-credentials

Sophia Zhou

Abstract

Modern life makes having a digital identity no longer optional. As the number of online services increases, it is necessary to safeguard user privacy and equip service providers with mechanisms enforcing Sybil resistance, i.e., preventing a single entity from showing as many.

To address this challenge, Alupotha et al. [1] introduce a new cryptographic notion, Anonymous Self-Credentials (ASC) along with two implementations. This approach enables users to maintain their privacy within an anonymity set while allowing SPs to obtain Sybil resistance.

In this work, we investigate how linkable ring signatures (LRS) may be used to build an ASC protocol. In particular, we design a LRS-based ASC protocol and benchmark LRS-based ASC against the ASC implementations in the original paper. We find that the LRS schemes have comparable, but typically slightly worse, size and performance as the original instantiations. This suggests that the original implementation of ASC may still be more practical, though LRS could be used as an alternative implementation of ASC.

1 Introduction

We live in a world with highly connected and ubiquitous digital ecosystems. Countless service providers (SPs) today rely on digital identities in their business models and functions, from social media apps to banks to online shopping platforms. Having a digital identity has become effectively unavoidable.

To ensure reliable and fair service, SPs must be able to defend against Sybil attacks, wherein a single user creates multiple identities to exploit, abuse, or manipulate the offered services, such as in a DDoS attack. We would like to uniquely map a each user to a Sybil-resistant identity, an identity that the service provider can assume represents a unique user. In practice, SPs often tie users to a phone number or email address, which although are not strictly Sybil-resistant, are assumed to be scarce because of the difficulty of obtaining or managing multiple of them.

However, users who use the same email or phone number to sign up for different services create a clear link between their profiles. Recently, as user data has become a valuable asset, user-tracking and sharing of user data between colluding SPs has become a well-known issue, often leading to violations of privacy in various ways. Thus, an identity authentication system that preserves unlinkability and user anonymity is very valuable.

To address both of these problems, Alupotha et al. [1] introduce a new cryptographic notion, Anonymous Self-Credentials (ASC), along with two implementations. This approach enables users to maintain their privacy within an anonymity set, a set of users such that the SP is unable to tell who in the set is actually signing in, while allowing SPs to obtain Sybil resistance. This self-derived credentials protocol offers unforgeability, robustness, Sybil resistance, anonymity, and multi-verifier unlinkability, and improves upon existing methods that are typically constructed from protocols requiring a central, trusted identity provider, such as group signatures and attribute-based credentials.

Our contribution. Linkable ring signatures [2] are cryptographic primitives that allow a user to anonymously sign on a message from within a set of users, but restrict users to only sign once on a given message. Thus, they seem like natural potential building blocks for an alternative construction of an ASC protocol. In this work, we have investigated how LRS may be used to build an ASC protocol.

First, we present an alternative construction of ASC using LRS, and observe that there is a clean mapping of the LRS interface onto ASC. Next, we benchmark several implementations of LRS and related primitives, such as traceable ring signatures (TRS) [3], which can similarly be used to construct ASC. Finally, we compare the performance of these LRS-based ASC implementations against the two ASC implementations of the original ASC paper.

2 Background

Linkable ring signatures. Digital signatures are a cryptographic notion that allow a signer with a public and secret key to produce a signature for a message that anyone with the signer’s public key can later verify was produced by the signer and not modified in any way [4]. Specifically, a digital signature scheme consists of three efficient methods: $\text{Gen}() \rightarrow (\text{pk}, \text{sk})$, $\text{Sign}(m, \text{sk}) \rightarrow \sigma$, $\text{Verify}(m, \sigma, \text{pk}) \rightarrow 0/1$. $\text{Gen}()$ is called by a user and returns a public and secret key generated according to a key generation scheme. $\text{Sign}(m, \text{sk})$ takes in a message and secret key and generates a signature. $\text{Verify}(m, \sigma, \text{pk})$ returns 0 if σ was not generated by the owner of pk and returns 1 if it was.

The security and correctness of the scheme is achieved through a zero-knowledge proof of knowledge (ZKPoK), a mathematical proof that the signer has secret knowledge without revealing to the verifier what the secret knowledge is. In the case of digital signatures, the signer proves to the verifier that they know the secret key without revealing the key itself. Digital signatures are widely used today in applications ranging from secure document signing, email signing, general identity and multi-factor authentication, and cryptocurrencies.

Ring signatures are a type of digital signature that allow a signer to select a list of users and generate a digital signature on a message which a verifier can later only confirm was generated by someone in the selected list of users [5]. Specifically, a ring signature scheme provides the following interface: $\text{Gen}() \rightarrow (\text{pk}, \text{sk})$, $\text{Sign}(m, \text{pk}_N, i, \text{sk}_i) \rightarrow \sigma$, $\text{Verify}(m, \sigma, \text{pk}_N) \rightarrow 0/1$. In this case, $\text{Sign}(m, \text{pk}_N, i, \text{sk}_i)$ takes in a message, a list of public keys, and an index i that identifies which member of the list the signer is, as well as the signer’s secret key, sk_i . $\text{Verify}(m, \sigma, \text{pk}_N)$ takes in a message, a signature, and a list of public keys, and outputs 1 if the signature was generated by someone with a public key in that list, and 0 otherwise. Ring signatures have applications in whistleblowing, anonymous voting, and anonymous transactions.

However, in some cases, generic ring signatures may provide too much anonymity. For example, when applied to a voting system where a message is a vote of yes or no, the above construction is unable to detect double-voting. To account for these cases, Liu et al. [6] introduced linkable spontaneous anonymous group signatures, or linkable ring signatures (LRS), an extension of ring signatures that restrict users to only be able to sign on a given message, or on a given public key list, once. LRS provides the following interface: $\text{Gen}() \rightarrow (\text{pk}, \text{sk})$, $\text{Sign}(m, \text{pk}_N, i, \text{sk}_i) \rightarrow (\sigma, \text{tag})$, $\text{Verify}(m, \sigma, \text{tag}, \text{pk}_N) \rightarrow 0/1$. As we can see, this interface is very similar to that of generic ring signatures, besides the additional generation of a tag during signing and input of a tag during verifying. This tag is a function of the signer’s secret key and the message or public key list. To link two signatures, we may simply check whether their tags are the same and return “linked” if they are.

As such, providing both anonymity and linkability, LRS seems like a natural choice of primitive for building an ASC protocol.

Traceable ring signatures are a further extension of LRS where in the case of signing two times on the same “issue,” we are able to trace the exact identity of the signer from their signatures [3]. TRS provides stronger functionality than is needed for ASC, but it is informative to see where, compared to LRS, the additional implementation overhead of TRS originates from, so a vanilla TRS scheme is among the schemes that we benchmarked.

Literature survey. There exist a number of LRS schemes in the literature, with a focus on achieving sublinear signing and verification times and proof sizes. However, the most widely used ring signature schemes to date are still those with performance and size linear in ring size, in other words, the original vanilla instantiations, as providing sublinear guarantees often generates significant additional overhead [7].

LRS schemes rely on a variety of assumptions, such as security in the random oracle model, discrete log hardness, decisional and symmetric external Diffie-Hellman hardness, existence of trapdoor one way permutations, and strong RSA. We list the assumptions of our five chosen LRS schemes below.

A common generic method for generating short linkable ring signatures is to use an accumulator to accumulate the public keys and then generate a proof that the signer’s public key has been accumulated. Ad Hoc Anonymous Identification by Dodis et al. [8], and Concise linkable ring signatures (CLSAG) by Gooddell et al. [9] use such an accumulation scheme. However, from computing the intermediate values in the accumulation, the signing and verification time of CLSAG is superlinear. CryptoNote, a transaction protocol, and Monero, a cryptocurrency, notably use linkable ring signatures to anonymize their transactions. Focusing on achieving small signature size, Monero uses CLSAG in their Ring Confidential Transactions with a fixed ring size of 16.

Bilinear maps, relying on the symmetric external Diffie-Hellman (SXDH) assumption, have also been used to achieve sublinear size and time, as in DualRing and DualDory [10]. Though DualDory is asymptotically logarithmic in signature size, it has a large constant, detracting from its practical use for smaller ring sizes.

Several lattice-based LRS have also been developed, such as FS-LLRS, SALRS, and Raptor [11]. For example, in this work, we benchmarked Raptor, lattice-based linkable ring signature that achieves similar efficiency as the original ring signature scheme by Rivest et al.. The main building block of Raptor is a function called a “chameleon hash plus” function, a randomized, collision-resistant hash function that is equipped with a trapdoor. Similar to other constructions, this function is used to generate intermediate values for all users except the signer for which the signer can find a collision to reverse engineer the desired value for the signature.

Thus, there are a variety of benefits and tradeoffs for different LRS schemes. Ultimately, we decided on five diverse SOTA schemes to benchmark: vanilla TRS (DDH), vanilla LRS (DLOG), CLSAG for shorter signatures via accumulation (DDH), DualDory for sublinear signatures and verification via bilinear mappings (SXDH), and Raptor, a practical lattice-based LRS.

Anonymous self-credentials. Anonymous self-credentials are defined as the tuple of

functions, (ASC.Setup, ASC.Gen, ASC.Prove, ASC.Verify, ASC.Open), such that they satisfy the following properties:

- Unforgeability: The authentication can only be created by a prover who knows a secret key to a master identity.
- Robustness: Malicious provers who create identities resembling correct master identities cannot impede the authentication of honest provers.
- Sybil resistance: Any two authentications by the same prover to the same verifier will be linkable.
- Anonymity: Authentications conceal the master identity of a prover within the group, provided that at least two provers created identities honestly.
- Multi-verifier unlinkability: Any two authentications for two verifiers conceal whether they are from the same prover or two different provers, even if the verifiers collude, assuming that at least two provers are honest.

3 Methodology

Through literature search, we identified five SOTA ring signature schemes and found public implementations of the schemes on Github. They are Traceable Ring Signature (TRS) (2006), Linkable Ring Signature (LRS) (2004), Concise Linkable Ring Signature (CLSAG) (2019), DualDory (2023), and Raptor (2018). TRS and CLSAG are implemented in Rust; DualDory and LRS are implemented in Go; Raptor and U2SSO-dbpoe are implemented in C; and U2SSO-snark is implemented in JS.

We benchmarked signing time, verification time, and signature size for each scheme by calculating the mean and standard deviation for 10 timings of signature generation and verification as well as the mean for 10 measurements of signature size, which are approximately constant between runs. We compute these performance statistics for each scheme and for ring sizes $\{2^n\}_{n=3}^{10}$. As a warm-up, we run these benchmarks two times before tracking the results of the third run. The entire benchmarking script took 1 hour and 7 minutes. The benchmarks were run on an Apple M1 chip with 16GB of RAM.

In addition to benchmarking, we designed a LRS-based ASC protocol using Liu et al.’s LRS interface and sketched proofs for the ASC guarantees.

4 Results

4.1 LRS-based ASC construction

First, we construct a slightly modified ASC protocol from a slightly modified version of the Liu et al.'s LRS scheme. We note that the generic LRS interface comprised of LRS.Gen, LRS.Sign, and LRS.verify, can be applied in the same way as outlined below, except for ASC.Open, which is scheme dependent. The change that we make to Liu et al.'s scheme is that instead of taking $h = H_2(pk_N)$ in LRS.Sign, we take $h = H_2(pk_N || (\phi, v_l))$. Letting (ϕ, v_l) be the *eventlabel*, which WLOG can be assumed to have the same distribution as (ϕ, v_l) , this exact modification was noted in section 6.1 of Liu et al. [6] to be a secure and computable alternative method of generating h .

Recall the ASC protocol is defined as a tuple of the following algorithms such that they satisfy correctness, robustness, strong unforgeability, Sybil resistance, anonymity, and multi-verifier unlinkability. Here, we keep the function names the same, but we define their inputs and outputs slightly differently using the LRS interface:

- ASC.Setup(): sets up the groups from which we generate the public and secret keys and the two hash functions, H_1, H_2 , used during signing and verification. This function may take in a security parameter λ .

```
def ASC.Setup():  
    LRS.Setup()
```

- ASC.Gen() $\rightarrow$ (pk, sk): generates a public and secret key pair for a user. The public key pk is set as the master identity of the user and published to the IdR.

```
def ASC.Gen():  
    return LRS.Gen()
```

- ASC.Prove($(\phi, v_l), pk_N, i, sk_i$) $\rightarrow$ (σ, nul): given a pseudonym ϕ , a verifier name/identifier v_l , an anonymity set/list of the public keys of the ring pk_N , the index of the signer's public key in the list i , and the signer's secret key sk_i , this function generates a signature σ on the pseudonym-verifier pair as well as a tag/nullifier.

```
def ASC.Prove(phi, v_l, pk_N, i, sk_i):  
    return LRS.Sign(phi, v_l, pk_N, i, sk_i)
```

- ASC.Open($(\phi, v_l), nul, sk_i$) $\rightarrow$ 0/1: a prover can later check if the nullifier nul was generated by them. This is not a function for verifiers.

```
def ASC.Open(phi, v_l, nul, sk_i):  
    return H_2(phi, v_l) ** sk_i == nul
```

- $\text{ASC.Verify}((\phi, v_l), pk_N, (\sigma, nul)) \rightarrow 0/1$: anyone can later check that ϕ was generated by a user whose public key is in pk_N and that nul was generated by the same person.

```
def ASC.Verify((phi, v_l), pk_N, (sig, nul)):
    return LRS.Verify((phi, v_l), pk_N, (sig, nul))
```

As we can see, the LRS functions map exactly onto the ASC protocol, and using LRS bypasses the generation of a master identity entirely. The only differences in the instantiation above with the original instantiation are the omission of the master identity and common reference string in the above, as well as the usage of (ϕ, v_l) as a pair rather than separately, for example the original ASC.Open only takes in v_l without ϕ as a parameter.

The above protocol inherits the correctness and security of LRS, including insider-corruption unforgeability, linkable-anonymity with adversarially chosen keys, linkability, and non-slanderability with adversarially chosen keys [12]. Letting the master identities be the public keys, it is clear from their definitions that some of these properties are equivalent to some of the ASC security guarantees, such as robustness and unforgeability, but some properties must be analyzed more in depth. The formulation of these proofs will be an ongoing component of this research lasting longer than the completion of this thesis.

Since the U2SSO system suggested by Alupotha et al. is built on top of ASC primitives, the design of the U2SSO system need not change, except requiring that each user keep a dictionary of the pseudonym she has registered with each SP, so that she can perform ASC.Open as defined above.

4.2 Benchmarking results

In this section, we present the benchmark results of the five LRS schemes and compare them with the two original ASC implementations. In Figs. 4.1 to 4.3 we display plots of the mean signing time, verification time, and proof size (respectively) of the 10 runs of the sign and verify functions of different schemes as they vary by the size of the ring. We sometimes omit schemes from these plots if their performance is significantly worse and overshadows the trends in the other schemes. Additional plots containing all the schemes are displayed in appendix A.2. In the plots, the bolded gray and black lines indicate the two instantiations of ASC from Alupotha et al..

Additionally, in general, the standard deviations of the experiments were very small compared to the means, except in the case of DualDory, which exhibited variance around 1/2 the magnitude of the mean, so they have not been displayed in the plots. The comparatively high variance in DualDory’s runtime may be because of higher runtime variance in operations used in bilinear mappings.

Overall, the current ASC instantiations perform on the same order of magnitude or better than the most performant ring signature schemes, and there is no single ring signature scheme that outperforms ASC in all properties we measured. Because of the similarity of ASC and LRS protocols, it may be possible to introduce a new instantiation of LRS via an instantiation of ASC. We would like to investigate this further.

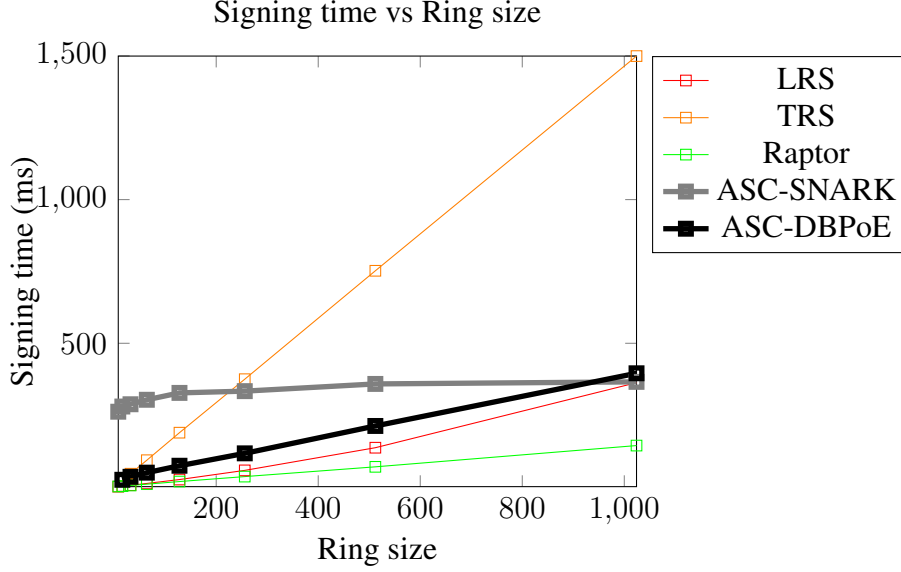


Figure 4.1: Signing time versus ring size.

There is high diversity in runtime and performance cost across different schemes. Since it is lattice-based, Raptor has low signing and verifying times but enormous size. CLSAG offers smaller signature sizes than other ring signatures at the cost of high signing and verifying times. DualDory offers logarithmic signature size but with a large constant, only achieving smaller signature size than TRS for some threshold greater than 1024 users.

We notice for any given LRS scheme, the original instantiations of ASC always perform better than LRS in signature size and verification time. Incidentally, we understand signature size and verification time to be the limiting factors of ASC, as we are sensitive to network and storage overhead when communicating and storing signatures, and SPs may have to execute many concurrent verifications, while each user comparatively infrequently registers, updates, or revokes their pseudonyms, which are the only instances where `ASC.Sign` is invoked. As in the real world, the effective number of SPs is assumed to be much smaller than the number of users (the total number of social media users is much bigger than the number of popular social media applications). Thus, the current instantiations of ASC may still be the most desirable way to implement a practical U2SSO system.

To date, the most widely used ring signatures are still those with size and timing linear in ring size, as sublinear ring signatures often present large overhead, as we can see from the performance of CLSAG and DualDory. However, in reality, runtime and memory cost may be more or less of a concern. Monero uses CLSAG in its Ring Confidential Transactions, as they do not need to use a large ring size and thus can be more indifferent about asymptotic runtime.

We also notice that LRS has slightly superlinear trends in signing and verifying time. This is likely because of the modular exponentiation that we do during the signing and verifying phases, where the exponents, on average, grow proportionally to ring size, and

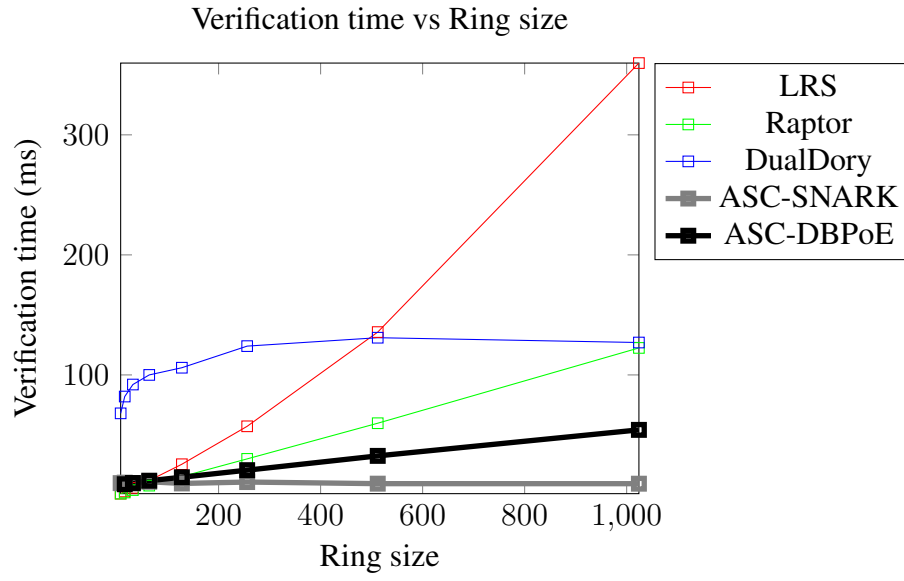


Figure 4.2: Verification time versus ring size.

indicates that this step should be reimplemented with constant-time exponentiation.

Thus, these results imply the original instantiations of ASC still achieve better practical performance than LRS, while LRS can be used to construct an alternative ASC protocol.

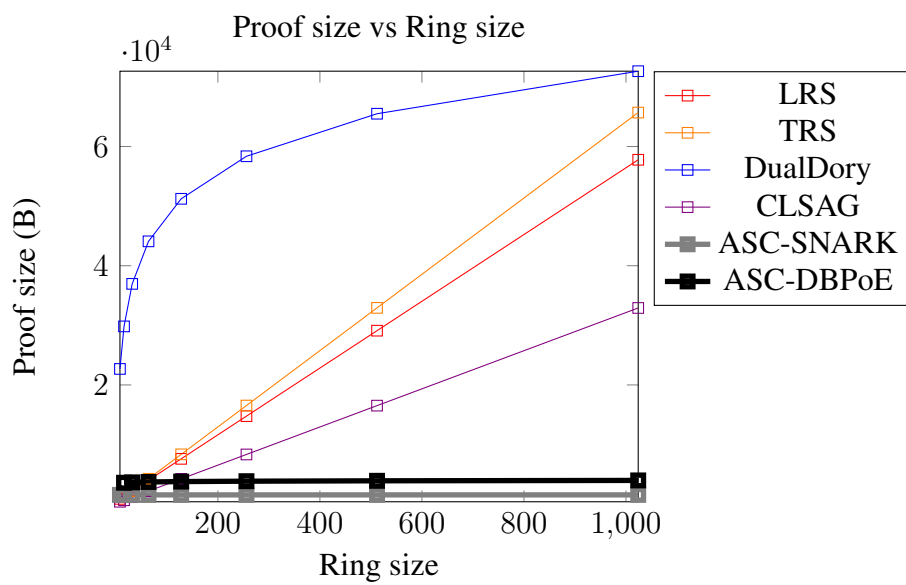


Figure 4.3: Proof size versus ring size.

5 Related Work

Existing privacy-preserving credentials are primarily constructed from anonymous credentials, group signatures, and generic ring signatures. In anonymous credentials, a user registers a master identity/secret key with an identity provider (IdP) and can create verifiable credentials on attributes and pseudonyms by querying the Issue function of the IdP. In group signatures, there is a trusted group manager, who collects public and secret keys and publishes the public keys to a trusted public board, along with a group of users, any of whom can sign a message without a verifier being able to tell who in the group signed the message [7]. Such a signature is called a “one-out-of-many” signature. However, the group manager has the special ability to check who generated any signature. Ring signatures also produce one-out-of-many signatures, but require no group manager and prior setup, and offer full anonymity, e.g. no one can check who created a signature. These schemes focus mainly on providing unlinkability, not Sybil resistance.

Recent efforts have been made to add Sybil resistance to IdP-based schemes using cryptographic techniques, such as using a function that produces only unique pseudonyms to check pseudonym equality [13].

However, IdP-based schemes have the drawback of relying on a central, trusted authority, which creates a single point of failure and the danger of deanonymization if the IdP and SPs were to collude, such as if they were to link which pseudonyms were registered with the IdP and SP at approximately the same time.

Instead of IdPs, self-sovereign identities use an immutable identity registry (IdR) to register pseudonyms which can be implemented with a decentralized ledger, like blockchains. One relevant example of an anonymous decentralized identity protocol is DID:RING which uses Borromean and Abe-Ohkubo-Suzuki ring signatures to generate anonymous authentications for digital subjects/users [14]. However, this and similar schemes still run into the problem of an untrusted IdR, which, in cases such as using the same payment method to register multiple pseudonyms, may still be able to link multiple pseudonyms. This is especially true for blockchain-based IdRs, where payment tracing is publicly available.

6 Conclusions

This work investigated the use of LRS to design and implement an ASC protocol. We found that there is an intuitive mapping between a generic LRS interface and the ASC protocol, and have used this notion to design a LRS-based ASC scheme as well as benchmark LRS.Sign and LRS.Verify against the signing and verifying functions of the original implementations. Our benchmarking results indicate that LRS-based ASC is practical, though performing slightly worse in the bottlenecks for an ASC system, verification time and proof size.

7 Future Work

We are interested in formally proving the security properties of LRS-based ASC. We may be interested in deriving an instantiation of LRS based on the original given instantiations of ASC. We may also be interested in developing a usable strawman implementation of a U2SSO system using LRS as the primitive.

Bibliography

- [1] Jayamine Alupotha et al. *Anonymous Self-Credentials and their Application to Single-Sign-On*. Cryptology ePrint Archive, Paper 2025/618. 2025. URL: <https://eprint.iacr.org/2025/618>.
- [2] Yuxi Xue et al. *Efficient Linkable Ring Signatures: New Framework and Post-Quantum Instantiations*. Cryptology ePrint Archive, Paper 2024/553. 2024. URL: <https://eprint.iacr.org/2024/553>.
- [3] Eiichiro Fujisaki and Koutarou Suzuki. *Traceable Ring Signature*. Cryptology ePrint Archive, Paper 2006/389. 2006. URL: <https://eprint.iacr.org/2006/389>.
- [4] Dan Boneh and Victor Shoup. *A Graduate Text in Applied Cryptography*. Earlier edition: 2023. Stanford Digital Repository, 2024. URL: <https://cryptobook.us>.
- [5] Ronald L. Rivest¹, Adi Shamir, and Yael Tauman. *How to Leak a Secret*. ASI-ACRYPT 2001, LNCS 2248, pp. 552–565. 2001. URL: <https://people.csail.mit.edu/rivest/pubs/RST01.pdf>.
- [6] Joseph K. Liu, Victor K. Wei, and Duncan S. Wong. *Linkable Spontaneous Anonymous Group Signature for Ad Hoc Groups*. Cryptology ePrint Archive, Paper 2004/027. 2004. URL: <https://eprint.iacr.org/2004/027>.
- [7] Alishah Chator, Matthew Green, and Pratyush Ranjan Tiwari. *SoK: Privacy-Preserving Signatures*. Cryptology ePrint Archive, Paper 2023/1039. 2023. URL: <https://eprint.iacr.org/2023/1039>.
- [8] Patrick P. Tsang and Victor K. Wei. *Short Linkable Ring Signatures for E-Voting, E-Cash and Attestation*. Cryptology ePrint Archive, Paper 2004/281. 2004. URL: <https://eprint.iacr.org/2004/281>.
- [9] Brandon Goodell, Sarang Noether, and Arthur Blue. *Concise Linkable Ring Signatures and Forgery Against Adversarial Keys*. Cryptology ePrint Archive, Paper 2019/654. 2019. URL: <https://eprint.iacr.org/2019/654>.
- [10] Jonathan Bootle et al. *DualDory: Logarithmic-Verifier Linkable Ring Signatures through Preprocessing*. Cryptology ePrint Archive, Paper 2023/1177. 2023. DOI: 10.1007/978-3-031-17146-8. URL: <https://eprint.iacr.org/2023/1177>.
- [11] Xingye Lu, Man Ho Au, and Zhenfei Zhang. *Raptor: A Practical Lattice-Based (Linkable) Ring Signature*. Cryptology ePrint Archive, Paper 2018/857. 2018. URL: <https://eprint.iacr.org/2018/857>.

- [12] Man Ho Au et al. “Short Linkable Ring Signatures Revisited”. In: *Public Key Infrastructure, EuroPKI 2006*. Ed. by Alessandro S. Atzeni and Alessandro Lioy. Vol. 4043. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2006. DOI: 10.1007/11774716_9. URL: https://doi.org/10.1007/11774716_9.
- [13] Maximilian Kroschewski, Anja Lehmann, and Cavit Özbay. *OPPID: Single Sign-On with Oblivious Pairwise Pseudonyms*. Cryptology ePrint Archive, Paper 2024/1124. 2024. URL: <https://eprint.iacr.org/2024/1124>.
- [14] Dimitrios Kasimatis et al. *DID:RING: Ring Signatures using Decentralised Identifiers For Privacy-Aware Identity*. 2024. arXiv: 2403.05271 [cs.CR]. URL: <https://arxiv.org/abs/2403.05271>.

A Appendix

A.1 Codebase

Here is the codebase that was used to generate the results: https://github.com/szhou0202/TRS_timing. Important note: the results generation for Raptor is currently not working through the automatic script, and to reproduce the results, you must set the NOU variable in Raptor/-param.h to each ring size you want to benchmark before running the test script, ./Raptor/bench.sh.

A.2 Additional plots

Figures A.1 to A.3 are additional plots of signing time, verification time, and signature size, respectively, including data for all seven implementations we benchmarked. We can see that CLSAG (purple) exhibits superlinear times in ring size. This is because the subroutine used to compute the challenges for all members during signing and verification is linear in ring size (it does a linear amount of scalar multiplications). This is the computation of the challenges for all members $W_i, \mathcal{W}_i$ in the signing algorithm.

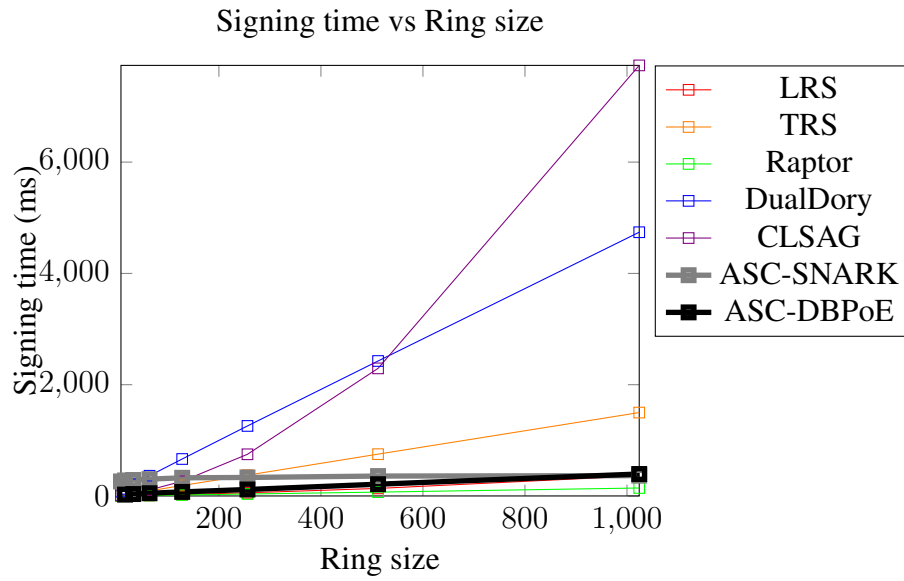


Figure A.1: Signing time versus ring size.

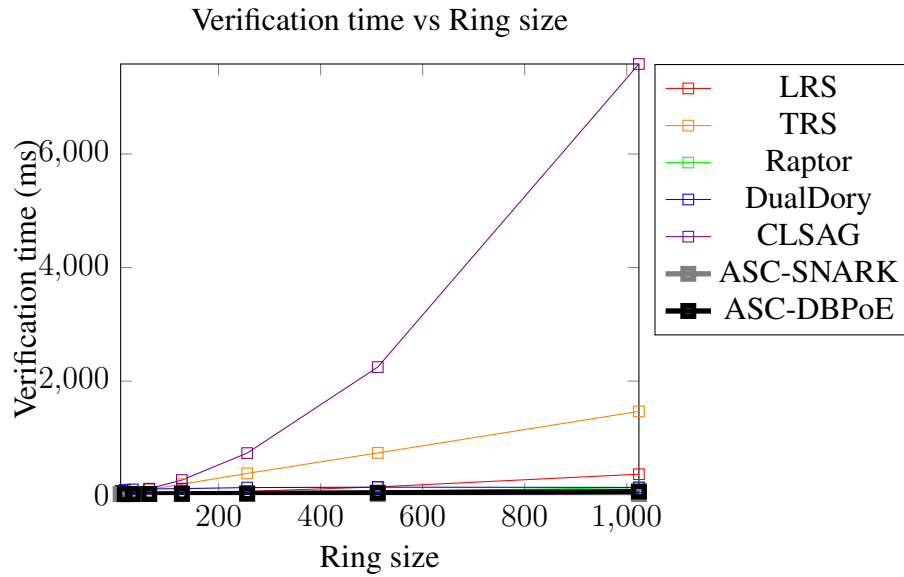


Figure A.2: Verification time versus ring size.

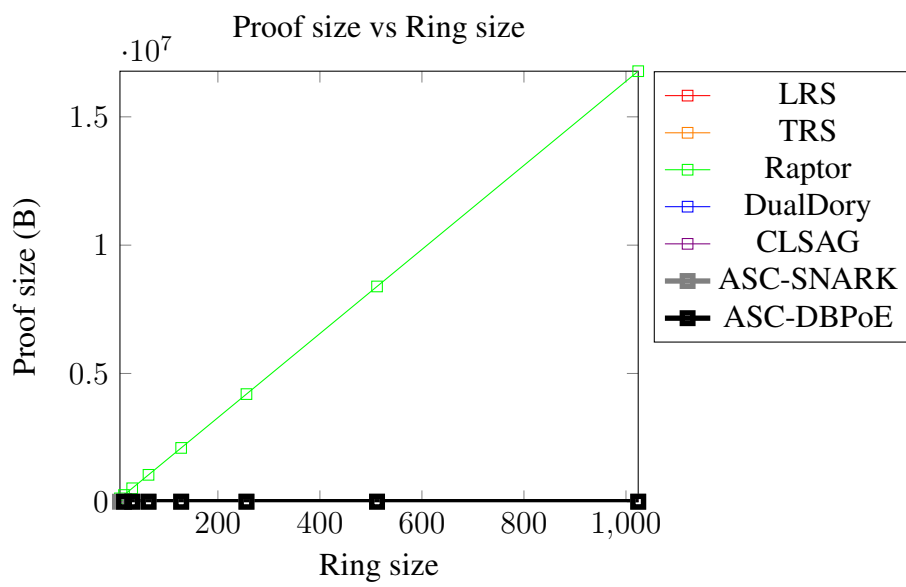


Figure A.3: Proof size versus ring size.